

IoT Platform Programming and Design for Smart Home Home Applications

**Internet of Things
Anthopoulou Elpida – Kalliopi
Argyriadou Anna**

1. Introduction & System Overview

This project implements a comprehensive **Smart Home IoT platform** designed for a real-time climate monitoring and automated environmental control. By integrating an **Arduino Uno** and a **DHT22 sensor** with **Node-RED**, the system provides a robust infrastructure for visualizing temperature, humidity, and the calculated heat index (Requirement A). Beyond real-time observation, the platform ensures data persistence through an **SQLite database** with 1-minute logging intervals (Requirement B) and automated hourly statistical aggregation (Requirement C). Finally, a sophisticated control loop is established to manage **threshold-based alerts** through both physical and network-based channels, ensuring a responsive and safe living environment (Requirement D).

System Overview & Architecture

The architecture follows a modular **three-layer** approach to ensure **stability** and **low-latency** performance. The **Perception Layer** utilizes an **Arduino edge gateway** to capture high-precision ambient telemetry and control a **local LED actuator**. This data is transmitted to the **Processing Layer**, where **Node-RED** serves as the **central orchestrator** for data parsing, logical evaluation, and feature engineering. For the **Application Layer**, a **web-based Dashboard** provides user-centric visualization, while a **serverless SQLite 3 engine** manages data persistence, facilitating efficient historical analytics and reporting directly on the host machine.

A core technical highlight of this platform is the implementation of a **hybrid communication model**. A strategically **Serial (UART)** communication was utilized for the **direct link** between **Node-RED** and the **Arduino** to ensure **minimal latency** for physical hardware feedback. By bypassing the network stack for logical signaling, the alert LED responds **instantly** to critical thresholds, **regardless** of network conditions. Complementing this, the **MQTT protocol** via a **local Mosquitto** broker provides a **scalable, asynchronous solution** for remote notifications. This dual-protocol approach ensures that the system remains **highly responsive** to local environment changes while staying **accessible** for future network expansions and multi-device integration.

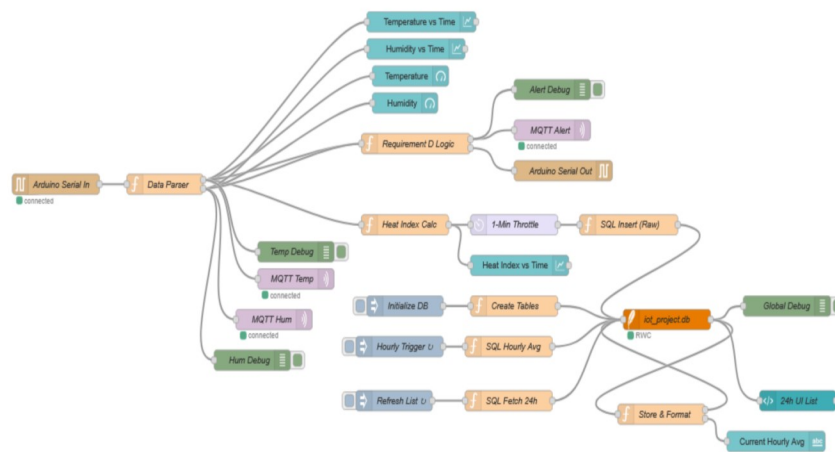


Figure 1: Node-RED Flow & Architecture

2. Real-Time Dashboard & Data Processing

The Visualization Layer of the platform is implemented through the **Node-RED Dashboard**, providing an intuitive interface for **real-time environmental monitoring**. To ensure data clarity, the system utilizes **radial gauges** for immediate temperature and humidity readings, while **line charts** track fluctuations over time, allowing the user to identify climate trends. The UI is designed to be responsive, fetching updates instantly as they are received from the edge device.

A critical component of the data Processing Layer is the calculation of the **Heat Index (HI)**, which represents the “apparent temperature” or how the environment actually feels to the human body. This is achieved through a **custom JavaScript function** node that parses the incoming serial string and applies the following physiological formula:

By integrating this metric into the dashboard, the platform provides a more **sophisticated assessment** of home comfort levels compared to simple raw sensor data.

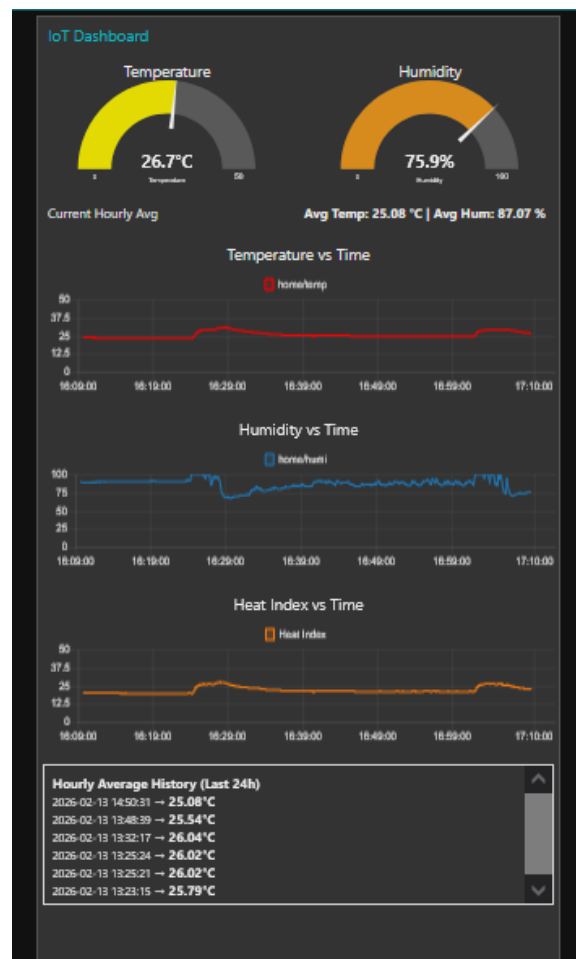
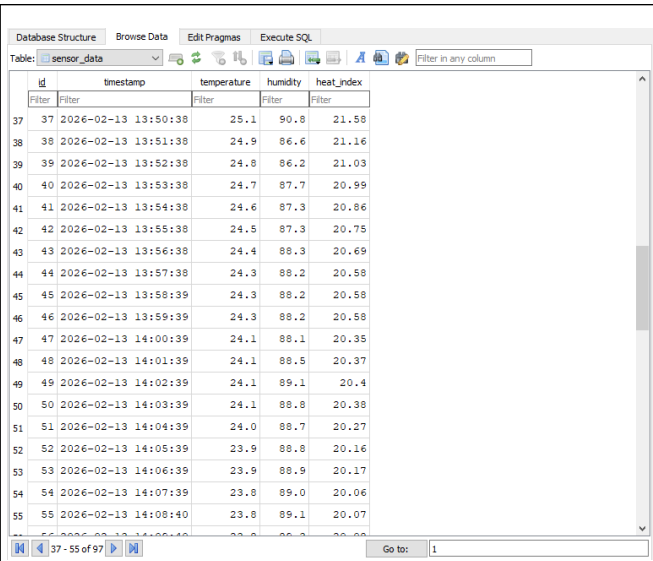


Figure 2: User Interface of IoT Platform

3. Database Integration & Historical Analytics

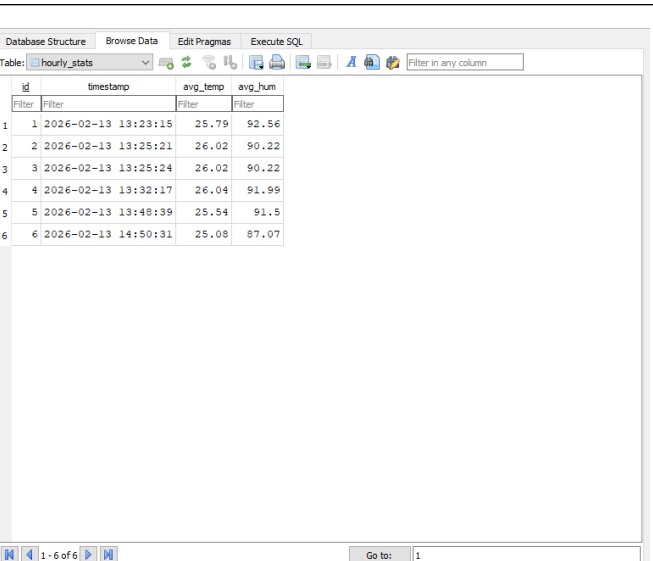
The system utilizes an **SQLite 3 engine** to ensure local data persistence, balancing lightweight performance with robust historical tracking. To fulfill Requirement B, telemetry data is **automatically** logged into the **sensor_data** table at 1-minute intervals. This frequency is managed via a **throttling mechanism** in Node-RED (**Delay node**), which captures the granular fluctuations of the environment while preventing unnecessary database overhead and ensuring long-term storage efficiency.

For Requirement C, the platform implements an **automated** Analytical Layer that **transforms** raw telemetry into meaningful historical insights. An **hourly trigger** executes an **SQL aggregation query**, calculating the arithmetic mean of temperatures and humidity recorded over the preceding 60 minutes using the **AVG()** function. The results are archived in the **hourly_stats** table and retrieved for visualization through a UI Template node. This approach provides the user with a structured, 24-hour historical log, essential for identifying broader climate patterns within the smart home ecosystem.



id	timestamp	temperature	humidity	heat_index
37	2026-02-13 13:50:38	25.1	90.8	21.58
38	2026-02-13 13:51:38	24.9	86.6	21.16
39	2026-02-13 13:52:38	24.8	86.2	21.03
40	2026-02-13 13:53:38	24.7	87.7	20.99
41	2026-02-13 13:54:38	24.6	87.3	20.86
42	2026-02-13 13:55:38	24.5	87.3	20.75
43	2026-02-13 13:56:38	24.4	88.3	20.69
44	2026-02-13 13:57:38	24.3	88.2	20.58
45	2026-02-13 13:58:39	24.3	88.2	20.58
46	2026-02-13 13:59:39	24.3	88.2	20.58
47	2026-02-13 14:00:39	24.1	88.1	20.35
48	2026-02-13 14:01:39	24.1	88.5	20.37
49	2026-02-13 14:02:39	24.1	89.1	20.4
50	2026-02-13 14:03:39	24.1	88.8	20.38
51	2026-02-13 14:04:39	24.0	88.7	20.27
52	2026-02-13 14:05:39	23.9	88.8	20.16
53	2026-02-13 14:06:39	23.9	88.9	20.17
54	2026-02-13 14:07:39	23.8	89.0	20.06
55	2026-02-13 14:08:40	23.8	89.1	20.07

Figure 3: sensor_data table from SQLite Database



id	timestamp	avg_temp	avg_hum
1	2026-02-13 13:23:15	25.79	92.56
2	2026-02-13 13:25:21	26.02	90.22
3	2026-02-13 13:25:24	26.02	90.22
4	2026-02-13 13:32:17	26.04	91.99
5	2026-02-13 13:48:39	25.54	91.5
6	2026-02-13 14:50:31	25.08	87.07

Figure 4: hourly_stats table from SQLite Database

4. Automation & Alert Mechanism

The platform features an intelligent control loop designed to respond to specific environmental risks. Using a **custom function node (Requirement D Logic)**, the system continuously evaluates the telemetry stream for a **combined** threshold condition: **Temperature** is **greater** than **30°C** and **Humidity** is **greater** than **70%**. This logic ensures that alerts are **only triggered when both parameters** indicate a high-discomfort or high-risk scenario, minimizing false positives.

To ensure reliability, the system employs a **hybrid feedback mechanism**. Once the criteria are met, a **direct command** is sent via the **Serial** interface to the **Arduino** to **activate** the physical **LED**, providing instant, low-latency local signaling. Simultaneously, is an **alert** payload is **published** to the **local MQTT** broker under the topic **home/alert**, enabling asynchronous notifications across the local network. The system **automatically resets** once conditions return to safe levels, maintaining a fully autonomous monitoring loop.

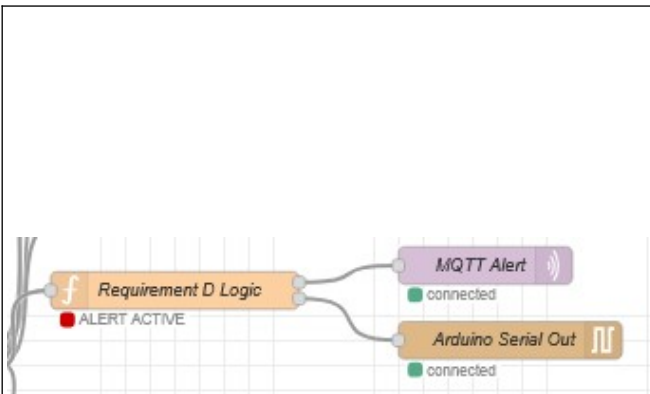


Figure 5: Requirement D Logic node sets alert status to ACTIVE

```
SELECT timestamp, avg_temp, avg_hum FROM hourly_stats ORDER BY timestamp
DESC LIMIT 24; : msg.payload : array[6]

▶ [ object, object, object, object, object, object ]

2/13/2026, 5:06:14 PM node: Alert Debug
home/alert : msg.payload : string[5]
"alert"

2/13/2026, 5:06:14 PM node: Alert Debug
home/alert : msg.payload : string[5]
"alert"

2/13/2026, 5:06:16 PM node: Global Debug
INSERT INTO sensor_data (temperature, humidity, heat_index, timestamp) VALUES (29.1,
72.8, 25.1... : msg.payload : array[0]
[ empty ]

2/13/2026, 5:06:19 PM node: Alert Debug
home/alert : msg.payload : string[9]
"stopalert"

2/13/2026, 5:06:20 PM node: Alert Debug
home/alert : msg.payload : string[9]
"stopalert"
```

Figure 6: Alert payload published on Mosquitto broker to topic home/alert

5. Summary & Results

The development IoT platform **successfully** integrates hardware sensing with **cloud-like middleware** and database persistence. All requirements (A, B, C and D) were met, demonstrating a **stable data flow** between the Arduino edge device and the Node-RED orchestrator. The system **effectively** handles real-time monitoring, automated data logging, and intelligent **threshold-based alerts** via Serial and MQTT protocols.

Appendices

Appendix A: Arduino Sketch

```
#include <DHT.h>

/* OPTIONAL NATIVE MQTT & WIFI IMPLEMENTATION (NON-ACTIVE)
 * The following code demonstrates the architecture for direct WiFi/MQTT
 * communication using the PubSubClient library.
 * * Note: This implementation was bypassed due to hardware limitations
 * NATIVE MQTT & WIFI IMPLEMENTATION (PROTOTYPE)
#include <WiFi.h>
#include <PubSubClient.h>

// Network & Broker Credentials
const char* ssid = "KorniliosXoirinos";
const char* password = "+DyoXeria3Podia@7-q";
const char* mqtt_server = "test.mosquitto.org";

WiFiClient espClient;
PubSubClient client(espClient);

void setup_wifi() {
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
}

// Requirement D: Handle incoming 'alert' and 'stopalert' signals from Node-RED
void callback(char* topic, byte* payload, unsigned int length) {
  String message;
  for (int i = 0; i < length; i++) message += (char)payload[i];

  if (String(topic) == "home/alert") {
    if (message == "alert") digitalWrite(LED_PIN, HIGH);    // Trigger LED ON
    else if (message == "stopalert") digitalWrite(LED_PIN, LOW); // Trigger LED OFF
  }
}

void reconnect() {
  while (!client.connected()) {
    if (client.connect("SmartHomeClient")) {
      client.subscribe("home/alert"); // Subscribe to the alert topic
    } else {
      delay(5000);
    }
  }
}
```

```

    }
}

// Requirement A & B: Publish raw data to dedicated MQTT topics
void publishData(float t, float h) {
    if (client.connected()) {
        client.publish("home/temp", String(t).c_str());
        client.publish("home/hum", String(h).c_str());
    }
}

*/
#define DHTPIN 2
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

const int alertLedPin = 13;
unsigned long lastExecutionMillis = 0;
const unsigned long interval = 5000;

void setup() {
    Serial.begin(9600);
    dht.begin();
    pinMode(alertLedPin, OUTPUT);
}

void loop() {
    // RECEIVE COMMANDS FROM NODE-RED
    if (Serial.available() > 0) {
        String command = Serial.readStringUntil('\n');
        command.trim();

        if (command == "LED_ON") {
            digitalWrite(alertLedPin, HIGH);
            Serial.println("ACK: ALERT_ACTIVE");
        }
        else if (command == "LED_OFF") {
            digitalWrite(alertLedPin, LOW);
            Serial.println("ACK: ALERT_STOPPED");
        }
    }

    // SEND TELEMETRY
    unsigned long currentMillis = millis();
    if (currentMillis - lastExecutionMillis >= interval) {
        lastExecutionMillis = currentMillis;
    }
}

```



```

float h = dht.readHumidity();
float t = dht.readTemperature();

if (!isnan(h) && !isnan(t)) {
  Serial.print("Temp: "); Serial.print(t);
  Serial.print(" Hum: "); Serial.println(h);
}
}
}

```

Appendix B: Node-RED JSON

```

[
  {
    "id": "a288cd4397f62ae2",
    "type": "tab",
    "label": "IoT Assignment",
    "disabled": false,
    "info": "",
    "env": []
  },
  {
    "id": "81bbca9dba5fba54",
    "type": "serial in",
    "z": "a288cd4397f62ae2",
    "name": "Arduino Serial In",
    "serial": "ffe0bd4863d1372e",
    "x": 110,
    "y": 300,
    "wires": [
      [
        "33747768e206ffaa"
      ]
    ]
  },
  {
    "id": "33747768e206ffaa",
    "type": "function",
    "z": "a288cd4397f62ae2",
    "name": "Data Parser",
    "func": "// Requirement A: Extracting and normalizing data from Serial input\nvar input = msg.payload.toString();\n\n// Using Regex to match Temperature and Humidity patterns from the Arduino string\nvar tMatch = input.match(/Temp:\\s*([\\d.]+)/);\nvar hMatch = input.match(/Hum:\\s*([\\d.]+)/);\n\nif (tMatch && hMatch) {\n  var t = parseFloat(tMatch[1]);\n  var h =

```

```

parseFloat(hMatch[1]);\n\n    // Storing values in flow context for global accessibility within the flow\n    flow.set('currentTemp', t);\n    flow.set('currentHumi', h);\n\n    // Returning separate messages for MQTT topics and Dashboard\n    gauges\n    return [\n        { topic: \"home/temp\", payload: t },\n        { topic: \"home/humi\", payload: h }\n    ];\n\n    \nreturn null; // Ignore malformed strings",

    "outputs": 2,

    "timeout": "",

    "noerr": 0,

    "initialize": "",

    "finalize": "",

    "libs": [],

    "x": 310,

    "y": 300,

    "wires": [

        [

            "ec36f316b6844e4c",

            "b01edc950829c3e3",

            "3ba13e42a7edf4c6",

            "c220efb16db2e43b",

            "eba434230e969b6f",

            "1ee70b9c4a5ff679"

        ],

        [

            "6bbbf687d6080906",

            "4c50b11df92c993d",

            "c220efb16db2e43b",

            "2e70d2dd8bfcc81c",

            "3a3199fc62649566"

        ]

    ]

},

{

    "id": "3ba13e42a7edf4c6",

    "type": "function",

    "z": "a288cd4397f62ae2",

    "name": "Heat Index Calc",

    "func": "// Calculate HI\nvar t = msg.payload;\nvar h = flow.get('currentHumi') || 0;\nvar hi = 0.5 * (t + 61.0 + ((t -\n68.0) * 1.2) + (h * 0.094));\nmsg.payload = parseFloat(hi.toFixed(2));\nmsg.topic = \"Heat Index\";\nreturn msg;",

    "outputs": 1,

    "timeout": "",

    "noerr": 0,

    "initialize": "",

```

```

    "finalize": "",
    "libs": [],
    "x": 750,
    "y": 360,
    "wires": [
      [
        "a15cef09477d8754",
        "5a425ffa5737bf8c"
      ]
    ]
  },
  {
    "id": "f892a7e3ee4bec2b",
    "type": "function",
    "z": "a288cd4397f62ae2",
    "name": "SQL Insert (Raw)",
    "func": "// Requirement B: Persistent Data Logging\nvar t = flow.get('currentTemp');\nvar h = flow.get('currentHumi'); //
Retrieving humidity from flow context\nvar hi = msg.payload; // Heat Index from the previous node\n\n// Constructing
the SQL INSERT statement with auto-timestamp\nmsg.topic = \"INSERT INTO sensor_data (temperature, humidity, heat_index,
timestamp) VALUES (\" \n          + t + \", \" + h + \", \" + hi + \", CURRENT_TIMESTAMP);\";\n\nreturn msg;\"",
    "outputs": 1,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1160,
    "y": 360,
    "wires": [
      [
        "b7a26365399fd0e3"
      ]
    ]
  },
  {
    "id": "c220efb16db2e43b",
    "type": "function",
    "z": "a288cd4397f62ae2",
    "name": "Requirement D Logic",
    "func": "// Get values from flow memory\nvar t = flow.get('currentTemp') || 0;\nvar h = flow.get('currentHumi') ||
0;\n\n// Requirement D Logic (Adjusted for testing)\nif (t > 30 && h > 70) {\n  node.status({fill:\\"red\\", shape:\\"dot\\", text:
\\\"ALERT ACTIVE\\\"});\n  return [\n    { payload: \\\"alert\\", topic: \\\"home/alert\\\" }, // Output 1 -> MQTT\n    { payload:"
```

```

\LED_ON\ } // Output 2 -> Serial\n ];\n} else {\n node.status({fill:\green\, shape:\ring\, text:
\Normal\});\n return [\n { payload: \stopalert\, topic: \home/alert\ }, // Output 1 -> MQTT\n { payload:
\LED_OFF\ } // Output 2 -> Serial\n ];\n}",

    "outputs": 2,

    "timeout": "",

    "noerr": 0,

    "initialize": "",

    "finalize": "",

    "libs": [],

    "x": 770,

    "y": 240,

    "wires": [

        [

            "e43ff36bbea30bfff",

            "f678496a2861d2a4"

        ],

        [

            "23c850dce571e09b"

        ]

    ]

},

{

    "id": "f85a177c4fb10507",

    "type": "inject",

    "z": "a288cd4397f62ae2",

    "name": "Hourly Trigger",

    "repeat": "3600",

    "crontab": "",

    "once": false,

    "topic": "",

    "payload": "",

    "payloadType": "date",

    "x": 740,

    "y": 540,

    "wires": [

        [

            "b21f79af9a3a9236"

        ]

    ]

},

{

```

```

    "id": "b21f79af9a3a9236",
    "type": "function",
    "z": "a288cd4397f62ae2",
    "name": "SQL Hourly Avg",
    "func": "// Requirement C: Fetching average values for Temperature and Humidity\nmsg.topic = \"SELECT AVG(temperature) as\navgT, AVG(humidity) as avgH FROM sensor_data WHERE timestamp > datetime('now', '-1 hour');\";\n\nreturn msg;",
    "outputs": 1,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 940,
    "y": 540,
    "wires": [
      [
        "b7a26365399fd0e3"
      ]
    ]
  },
  {
    "id": "4156d6c4313c09b6",
    "type": "function",
    "z": "a288cd4397f62ae2",
    "name": "Store & Format",
    "func": "// Requirement C: Prevent NaN and stabilize Dashboard display\n// The database sends an empty array after every\nINSERT operation.\n// We must check for this and stop the flow to keep the Dashboard stable.\n\nif (!msg.payload ||\nmsg.payload.length === 0 || msg.payload[0].avgT === undefined) {\n  // Return null stops the empty message from reaching the\n  Dashboard\n  return null; \n}\n\n// If we reach here, it means we have valid data from the SELECT query\n\nvar t =\nparseFloat(msg.payload[0].avgT).toFixed(2);\nvar h = parseFloat(msg.payload[0].avgH).toFixed(2);\n\n// Output 1 (Top): For the\nSQLite node (INSERT)\n\nvar msgSql = { \n  topic: \"INSERT INTO hourly_stats (avg_temp, avg_hum, timestamp) VALUES (\" + t + \",\n\" + h + \", CURRENT_TIMESTAMP);\" \n};\n\n// Output 2 (Bottom): For the Dashboard Text node\n\nvar msgUi = { \n  payload: \"Avg\nTemp: \" + t + \" °C | Avg Hum: \" + h + \" %\" \n};\n\nreturn [msgSql, msgUi];",
    "outputs": 2,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1220,
    "y": 640,
    "wires": [
      [

```

```

        "b7a26365399fd0e3"
    ],
    [
        "7e4efe7343fd460e"
    ]
]
},
{
    "id": "56d16c0155ee6743",
    "type": "inject",
    "z": "a288cd4397f62ae2",
    "name": "Refresh List",
    "repeat": "60",
    "crontab": "",
    "once": true,
    "topic": "",
    "payload": "",
    "payloadType": "date",
    "x": 730,
    "y": 620,
    "wires": [
        [
            "be35b60e6dda69f5"
        ]
    ]
},
{
    "id": "be35b60e6dda69f5",
    "type": "function",
    "z": "a288cd4397f62ae2",
    "name": "SQL Fetch 24h",
    "func": "// Requirement C: Retrieving the last 24 hourly averages from the database\nmsg.topic = \"SELECT timestamp,\navg_temp, avg_hum FROM hourly_stats ORDER BY timestamp DESC LIMIT 24;\";\nreturn msg;",
    "outputs": 1,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 940,

```

```

        "y": 620,
        "wires": [
            [
                "b7a26365399fd0e3"
            ]
        ]
    },
    {
        "id": "bd81269cecc32477",
        "type": "ui_template",
        "z": "a288cd4397f62ae2",
        "group": "g1",
        "name": "24h UI List",
        "order": 10,
        "width": 12,
        "height": 5,
        "format": "<div style=\"height:150px; overflow-y:scroll; border:1px solid #ddd; padding:10px;\">\n    <b>Hourly Average  
History (Last 24h)</b>\n    <div ng-repeat=\"row in msg.payload\">\n        <small>{{row.timestamp}}</small> →  
<b>{{row.avg_temp}}°C</b>\n    </div>\n</div>",
        "storeOutMessages": true,
        "fwdInMessages": true,
        "resendOnRefresh": false,
        "templateScope": "local",
        "className": "",
        "x": 1470,
        "y": 620,
        "wires": [
            []
        ]
    },
    {
        "id": "d04dd847d7540808",
        "type": "inject",
        "z": "a288cd4397f62ae2",
        "name": "Initialize DB",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",

```

```

        "vt": "str"
    }
],
"repeat": "",
"crontab": "",
"once": false,
"onceDelay": "",
"topic": "",
"payload": "",
"payloadType": "str",
"x": 750,
"y": 480,
"wires": [
    [
        "ba6577d801d50504"
    ]
]
},
{
    "id": "ba6577d801d50504",
    "type": "function",
    "z": "a288cd4397f62ae2",
    "name": "Create Tables",
    "func": "// Requirement B & C: Initializing Database Schema\nvar msg1 = { \n    topic: \"CREATE TABLE IF NOT EXISTS\nsensor_data (id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp DATETIME DEFAULT CURRENT_TIMESTAMP, temperature REAL, humidity REAL,\nheat_index REAL);\n\" \n};\n\nvar msg2 = { \n    topic: \"CREATE TABLE IF NOT EXISTS hourly_stats (id INTEGER PRIMARY KEY\nAUTOINCREMENT, timestamp DATETIME DEFAULT CURRENT_TIMESTAMP, avg_temp REAL, avg_hum REAL);\n\" \n};\n\n// Return as an array of\nmessages to be executed sequentially\nreturn [ msg1, msg2 ] ;",
    "outputs": 1,
    "timeout": "",
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 950,
    "y": 480,
    "wires": [
        [
            "b7a26365399fd0e3"
        ]
    ]
}
]

```



```

},
{
  "id": "b7a26365399fd0e3",
  "type": "sqlite",
  "z": "a288cd4397f62ae2",
  "mydb": "db1",
  "sql": "",
  "name": "iot_project.db",
  "x": 1230,
  "y": 500,
  "wires": [
    [
      "75e1be43babf72ea",
      "4156d6c4313c09b6",
      "bd81269cecc32477"
    ]
  ]
},
{
  "id": "e43ff36bbea30bfff",
  "type": "mqtt out",
  "z": "a288cd4397f62ae2",
  "name": "MQTT Alert",
  "topic": "home/alert",
  "qos": "",
  "retain": "",
  "respTopic": "",
  "contentType": "",
  "userProps": "",
  "correl": "",
  "expiry": "",
  "broker": "mqtt_broker_local",
  "x": 1020,
  "y": 220,
  "wires": []
},
{
  "id": "23c850dce571e09b",
  "type": "serial out",
  "z": "a288cd4397f62ae2",

```

```

    "name": "Arduino Serial Out",
    "serial": "ffe0bd4863d1372e",
    "x": 1040,
    "y": 280,
    "wires": []
  },
  {
    "id": "ec36f316b6844e4c",
    "type": "ui_gauge",
    "z": "a288cd4397f62ae2",
    "name": "",
    "group": "g1",
    "order": 1,
    "width": 6,
    "height": 3,
    "gtype": "gage",
    "title": "Temperature",
    "label": "Temperature",
    "format": "{{value}}°C",
    "min": 0,
    "max": 50,
    "colors": [
      "#00b500",
      "#e6e600",
      "#ca3838"
    ],
    "seg1": "",
    "seg2": "",
    "diff": false,
    "className": "",
    "x": 760,
    "y": 140,
    "wires": []
  },
  {
    "id": "6bbbf687d6080906",
    "type": "ui_gauge",
    "z": "a288cd4397f62ae2",
    "name": "",
    "group": "g1",

```

```

    "order": 2,
    "width": 6,
    "height": 3,
    "gtype": "gage",
    "title": "Humidity",
    "label": "Humidity",
    "format": "{{value}}%",
    "min": 0,
    "max": 100,
    "colors": [
        "#00b500",
        "#e6e600",
        "#ca3838"
    ],
    "seg1": "",
    "seg2": "",
    "diff": false,
    "className": "",
    "x": 750,
    "y": 180,
    "wires": []
  },
  {
    "id": "7e4efe7343fd460e",
    "type": "ui_text",
    "z": "a288cd4397f62ae2",
    "group": "g1",
    "order": 3,
    "width": "",
    "height": "",
    "name": "",
    "label": "Current Hourly Avg",
    "format": "{{msg.payload}} ",
    "layout": "",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": "",
    "color": "#000000",
    "x": 1430,

```

```

    "y": 680,
    "wires": []
  },
  {
    "id": "b01edc950829c3e3",
    "type": "ui_chart",
    "z": "a288cd4397f62ae2",
    "name": "",
    "group": "g1",
    "order": 4,
    "width": 12,
    "height": "4",
    "label": "Temperature vs Time",
    "chartType": "line",
    "legend": "true",
    "xformat": "HH:mm:ss",
    "interpolate": "linear",
    "nodata": "",
    "dot": false,
    "ymin": "0",
    "ymax": "50",
    "removeOlder": "1",
    "removeOlderPoints": "",
    "removeOlderUnit": "3600",
    "cutout": "",
    "useOneColor": false,
    "useUTC": false,
    "colors": [
      "#ff0000",
      "#aec7e8",
      "#ff7f0e",
      "#2ca02c",
      "#98df8a",
      "#ff0000",
      "#ff9896",
      "#9467bd",
      "#c5b0d5"
    ],
    "outputs": 1,
    "useDifferentColor": false,

```

```

        "className": "",
        "x": 780,
        "y": 60,
        "wires": [
            []
        ]
    },
    {
        "id": "4c50b11df92c993d",
        "type": "ui_chart",
        "z": "a288cd4397f62ae2",
        "name": "",
        "group": "g1",
        "order": 5,
        "width": 12,
        "height": "4",
        "label": "Humidity vs Time",
        "chartType": "line",
        "legend": "true",
        "xformat": "HH:mm:ss",
        "interpolate": "linear",
        "nodata": "",
        "dot": false,
        "ymin": "0",
        "ymax": "100",
        "removeOlder": "1",
        "removeOlderPoints": "",
        "removeOlderUnit": "3600",
        "cutout": "",
        "useOneColor": false,
        "useUTC": false,
        "colors": [
            "#1f77b4",
            "#aec7e8",
            "#ff7f0e",
            "#2ca02c",
            "#98df8a",
            "#ff0000",
            "#ff9896",
            "#9467bd",

```

```

        "#c5b0d5"
    ],
    "outputs": 1,
    "useDifferentColor": false,
    "className": "",
    "x": 770,
    "y": 100,
    "wires": [
        []
    ]
},
{
    "id": "a15cef09477d8754",
    "type": "ui_chart",
    "z": "a288cd4397f62ae2",
    "name": "",
    "group": "g1",
    "order": 6,
    "width": 12,
    "height": "4",
    "label": "Heat Index vs Time",
    "chartType": "line",
    "legend": "true",
    "xformat": "HH:mm:ss",
    "interpolate": "linear",
    "nodata": "",
    "dot": false,
    "ymin": "0",
    "ymax": "50",
    "removeOlder": "1",
    "removeOlderPoints": "",
    "removeOlderUnit": "3600",
    "cutout": "",
    "useOneColor": false,
    "useUTC": false,
    "colors": [
        "#ff7f0e",
        "#aec7e8",
        "#ff7f0e",
        "#2ca02c",
    ]
}

```

```

        "#98df8a",
        "#d62728",
        "#ff9896",
        "#9467bd",
        "#c5b0d5"
    ],
    "outputs": 1,
    "useDifferentColor": false,
    "className": "",
    "x": 960,
    "y": 420,
    "wires": [
        []
    ]
},
{
    "id": "75e1be43babf72ea",
    "type": "debug",
    "z": "a288cd4397f62ae2",
    "name": "Global Debug",
    "active": true,
    "x": 1450,
    "y": 480,
    "wires": []
},
{
    "id": "5a425ffa5737bf8c",
    "type": "delay",
    "z": "a288cd4397f62ae2",
    "name": "1-Min Throttle",
    "pauseType": "rate",
    "timeout": "5",
    "timeoutUnits": "seconds",
    "rate": "1",
    "nbRateUnits": "1",
    "rateUnits": "minute",
    "randomFirst": "1",
    "randomLast": "5",
    "randomUnits": "seconds",
    "drop": true,

```

```

    "allowrate": false,
    "outputs": 1,
    "x": 950,
    "y": 360,
    "wires": [
      [
        "f892a7e3ee4bec2b"
      ]
    ]
  },
  {
    "id": "f678496a2861d2a4",
    "type": "debug",
    "z": "a288cd4397f62ae2",
    "name": "Alert Debug",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "payload",
    "targetType": "msg",
    "statusVal": "",
    "statusType": "auto",
    "x": 1020,
    "y": 160,
    "wires": []
  },
  {
    "id": "eba434230e969b6f",
    "type": "mqtt out",
    "z": "a288cd4397f62ae2",
    "name": "MQTT Temp",
    "topic": "home/temp",
    "qos": "",
    "retain": "",
    "respTopic": "",
    "contentType": "",
    "userProps": "",
    "correl": "",
    "expiry": "",

```



```

    "broker": "mqtt_broker_local",
    "x": 550,
    "y": 440,
    "wires": []
  },
  {
    "id": "2e70d2dd8bfcc81c",
    "type": "mqtt out",
    "z": "a288cd4397f62ae2",
    "name": "MQTT Hum",
    "topic": "home/hum",
    "qos": "",
    "retain": "",
    "respTopic": "",
    "contentType": "",
    "userProps": "",
    "correl": "",
    "expiry": "",
    "broker": "mqtt_broker_local",
    "x": 510,
    "y": 500,
    "wires": []
  },
  {
    "id": "1ee70b9c4a5ff679",
    "type": "debug",
    "z": "a288cd4397f62ae2",
    "name": "Temp Debug",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "payload",
    "targetType": "msg",
    "statusVal": "",
    "statusType": "auto",
    "x": 550,
    "y": 400,
    "wires": []
  },

```

```

{
  "id": "3a3199fc62649566",
  "type": "debug",
  "z": "a288cd4397f62ae2",
  "name": "Hum Debug",
  "active": true,
  "tosidebar": true,
  "console": false,
  "tostatus": false,
  "complete": "payload",
  "targetType": "msg",
  "statusVal": "",
  "statusType": "auto",
  "x": 470,
  "y": 560,
  "wires": []
},
{
  "id": "ffe0bd4863d1372e",
  "type": "serial-port",
  "name": "",
  "serialport": "COM3",
  "serialbaud": "9600",
  "databits": 8,
  "parity": "none",
  "stopbits": 1,
  "waitfor": "",
  "dtr": "none",
  "rts": "none",
  "cts": "none",
  "dsr": "none",
  "newline": "\\n",
  "bin": "false",
  "out": "char",
  "addchar": "\\n",
  "responsetimeout": 10000
},
{
  "id": "g1",
  "type": "ui_group",

```

```

    "name": "IoT Dashboard",
    "tab": "t1",
    "order": 1,
    "disp": true,
    "width": "12",
    "collapse": false,
    "className": ""
  },
  {
    "id": "db1",
    "type": "sqlitedb",
    "db": "C:\\Users\\Solringen\\Desktop\\Μεταπτυχιακό\\1ο Εξάμηνο\\Internet of Things\\Assignment\\iot_project.db",
    "mode": "RWC"
  },
  {
    "id": "mqtt_broker_local",
    "type": "mqtt-broker",
    "name": "Local Mosquitto",
    "broker": "127.0.0.1",
    "port": "1883",
    "clientid": "",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": 4,
    "keepalive": 15,
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthPayload": "",
    "birthMsg": {},
    "closeTopic": "",
    "closePayload": "",
    "closeMsg": {},
    "willTopic": "",
    "willQos": "0",
    "willPayload": "",
    "willMsg": {},
    "userProps": "",
    "sessionExpiry": ""
  }

```

```
},  
{  
  "id": "t1",  
  "type": "ui_tab",  
  "name": "Home",  
  "icon": "dashboard"  
},  
{  
  "id": "033b3a678dd1aa91",  
  "type": "global-config",  
  "env": [],  
  "modules": {  
    "node-red-node-serialport": "2.0.3",  
    "node-red-dashboard": "3.6.6",  
    "node-red-node-sqlite": "1.1.1"  
  }  
}  
]
```